

MÁQUINA DE ESTADOS (FERREIRO E MINEIRO)

1. INTRODUÇÃO

O foco central dessa atividade está na simulação de rotina e na dependência entre dois agentes autônomos em um ambiente controlado via console Java, utilizando as classes Main.java e AgentManager.java. A proposta busca realizar comportamentos com interações diretas de necessidade, como a extração de minério pelo Agente A (Mineiro) e a necessidade de minério para forjar espadas do Agente B (Ferreiro).

2. AGENTES

2.1 Agente A (Mineiro)

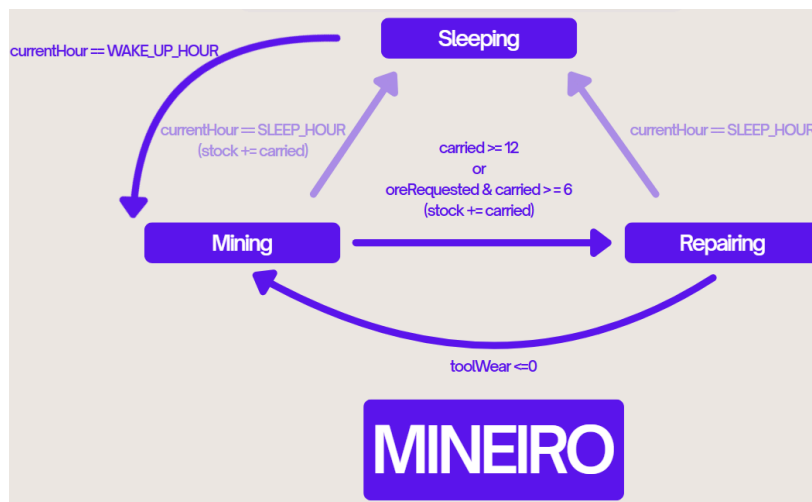
O Agente A (Mineiro) atua como o provedor, sendo responsável por minerar e manter a picareta em condições de uso. Ele inicia seu ciclo dormindo (Sleeping), até que chega seu horário de acordar (WAKE_UP_HOUR) e entra no estado de minerar. Conforme minera, ele acumula minério na mochila (carried) e desgasta a picareta (toolWear). Ao atingir o valor necessário de minério (MINING_DONE com valor 12), Mineiro guarda o minério no estoque (stock), o que avisa ao Ferreiro que há material disponível, e troca para o estado Repairing, onde repara a picareta antes de reiniciar todo o processo.

2.2 Agente B (Ferreiro)

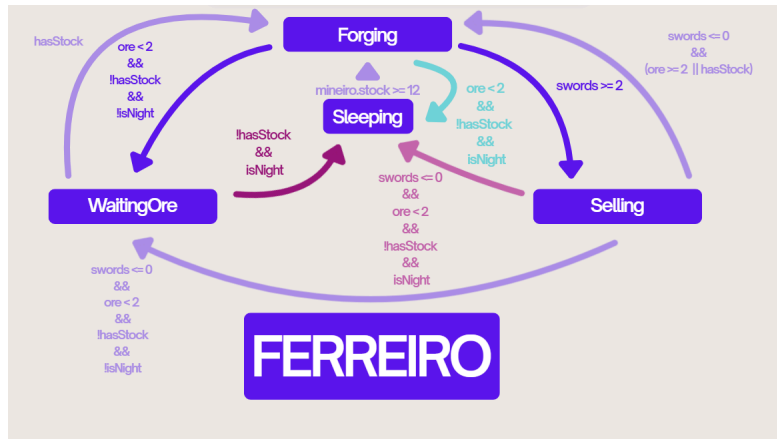
O Agente B (Ferreiro) possui o seu comportamento guiado pela quantidade de minério (ore) e pelo número de espadas prontas (swords), iniciando no estado Sleeping até que 12 minérios estejam disponíveis no estoque, já que começa sem nenhum minério. Quando consegue pegar o estoque do Mineiro, ele vai para o estado Forging, onde cada hora gasta 2 de minério e avança o progresso da espada. Quando o progresso chega a SWORD_DONE (20), uma espada fica pronta. Ao juntar SWORDS_TO_SELL (2) espadas, Ferreiro muda para o estado Selling e vende uma espada por hora, ganhando ouro. Se o minério acabar durante a forja e o Mineiro não tiver estoque, ele entra em WaitingOre, envia um pedido ao Mineiro e espera o minério chegar.

3. DIAGRAMAS

3.1 Mineiro



3.2 Ferreiro



4. TABELA DE REGRAS

4.1 Agente A (Mineiro)

Estado atual	Ação de entrada	Ação de execução	Ação de saída	Condição de transição
Mining	Mostra no console a mensagem "Vamos minerar".	Executa a função mineiro.mine() , que aumenta o carried em 3 e o toolWear em 2, imprimindo o status "Minerando...".	Executa storeOre() , guarda no estoque e avisa no console "O minério foi guardado no estoque para o ferreiro!"	Quando carried for maior ou igual a MINING_DONE (12), ou quando oreRequested for true e carried for maior ou igual a MIN_FOR_REQUEST (6), altera o estado para Repairing . SE currentHour == SLEEP_HOUR (20:00), muda para Sleeping .
Repairing	Exibe no console "A picareta está gasta, preciso reparar...".	Executa a função mineiro.repair() , reduzindo o toolWear em 4 (com valor mínimo 0), e imprime o status "Reparando a picareta...".	Exibe no console a mensagem "Picareta reparada!".	Quando toolWear for menor ou igual a 0, altera o estado de volta para Mining . SE currentHour == SLEEP_HOUR (20:00), muda para Sleeping .
Sleeping	Exibe "MINEIRO foi dormir...".	Imprime "Dormindo...".	Exibe "MINEIRO acordou!".	Quando currentHour for igual a WAKE_UP_HOUR (05:00), muda para Mining .

4.2 Agente B (Ferreiro)

Estado atual	Ação de entrada	Ação de execução	Ação de saída	Condição de transição
Forging	Exibe no console "Forja acesa!".	Executa a função forge() , que diminui o ore em 2 e aumenta o progress em 5. Quando o progress chega a 20, ele volta para 0 e swords aumenta em 1. Imprime "Forjando...".	Não possui ação específica.	Se swords >= 2 muda para Selling . Caso contrário, se ore < 2 e refill() não conseguir pegar estoque, muda para WaitingOre (ou para Sleeping se atingir NIGHT_HOUR).

WaitingOre	Mostra a mensagem "Acabou o minério! Mineiro, preciso de mais!" e executa askForOre() , que define oreRequested como true no Mineiro .	Imprime "Esperando o minério..." sem alterar nenhum valor.	Exibe no console "O minério chegou!".	Se refill() conseguir pegar o estoque do Mineiro , muda para o estado Forging . Se IsNight() && ore < 2 && !hasStock muda para o estado Sleeping .
Selling	Mostra a mensagem "Espadas prontas, hora de vender na feira!".	Executa a função sell() , que diminui swords em 1 e aumenta gold em 10, imprimindo "Vendendo...".	Não possui ação específica.	Quando swords $\leq$ 0, se houver minério (próprio ou no estoque), muda para Forging . Se ore < 2 e !hasStock (sem minério disponível), muda para WaitingOre ou se for IsNight() muda para Sleeping .
Sleeping	Exibe "FERREIRO foi dormir...".	Imprime "Dormindo...".	Exibe "FERREIRO acordou!".	Quando mineiro.stock $\geq$ 12, executa refill() e muda para Forging .

5. VARIÁVEIS E LIMIARES POR AGENTE

5.1 Agente A (Mineiro)

- **MINING_DONE**: Constante inteira com valor 12. É a quantidade de minério na mochila que faz Mineiro parar de minerar e ir reparar a picareta.
- **MIN_FOR_REQUEST**: Constante inteira com valor 6. É a quantidade mínima de minério que Mineiro precisa ter na mochila para atender um pedido antes de chegar a 12.
- **WAKE_UP_HOUR**: Constante inteira com valor 5. Define o horário (05:00) em que o Mineiro acorda do estado **Sleeping** e começa o trabalho.
- **SLEEP_HOUR**: Constante inteira com valor 20. Define o horário (20:00) em que o Mineiro encerra o expediente, guarda o minério recolhido no estoque e vai dormir.
- **carried**: Variável inteira iniciada em 0. Aumenta de 3 em 3 durante o estado Mining e volta para 0 quando o minério é guardado no estoque.
- **stock**: Variável inteira iniciada em 0. Recebe o minério da mochila quando Mineiro sai de Mining e volta para 0 quando o Ferreiro pega o minério.
- **toolWear**: Variável inteira iniciada em 0. Aumenta em 2 a cada hora de mineração e diminui em 4 a cada hora no estado Repairing, tendo 0 como valor mínimo.
- **oreRequested**: Variável booleana iniciada em false. Passa para true quando o Ferreiro entra em WaitingOre e volta para false quando o Ferreiro pega o estoque..

5.2 Agente B (Ferreiro)

- **ORE_TO_START_WORK**: Constante inteira com valor 12. É a quantidade mínima de minério necessária no estoque para fazer o Ferreiro acordar e iniciar o seu expediente.
- **NIGHT_HOUR / NIGHT_END_HOUR**: Constante com a definição de noite [20 (NIGHT_HOUR) até as 05 (NIGHT_END_HOUR)].

- **ORE_PER_CYCLE:** Constante inteira com valor 2. É o minério gasto a cada hora de forja, e também o mínimo que Ferreiro precisa ter para continuar forjando.
- **SWORD_DONE:** Constante inteira com valor 20. É o valor que progress precisa atingir para uma espada ficar pronta.
- **SWORDS_TO_SELL:** Constante inteira com valor 2. É o número de espadas prontas que faz Ferreiro ir para o estado Selling.
- **ore:** Variável inteira iniciada em 0. Aumenta quando Ferreiro pega o estoque do Mineiro e diminui em 2 a cada hora no estado Forging.
- **progress:** Variável inteira iniciada em 0. Aumenta em 5 a cada hora de forja e volta para 0 quando uma espada fica pronta.
- **swords:** Variável inteira iniciada em 0. Aumenta em 1 quando uma espada fica pronta e diminui em 1 a cada hora no estado Selling.
- **gold:** Variável inteira iniciada em 0. Aumenta em 10 a cada espada vendida.
- **mineiro:** Referência que conecta Ferreiro ao agente Mineiro, permitindo checar o estoque (`hasStock()`), pegar o minério (`takeStock()`) e enviar o pedido (`requestOre()`).

6. ESTRUTURA DO CÓDIGO

1. **State.java:** Interface que garante que qualquer estado saiba exatamente o que fazer quando o agente entra nele, o que faz enquanto permanece nele e o que faz ao sair dele.
2. **Agent.java:** Classe abstrata base para os agentes. Guarda o nome do agente e a referência do estado atual, além de implementar o método `changeState()` que executa a saída do estado antigo, atualiza a referência e chama a entrada do novo estado.
3. **Mineiro.java:** Classe do agente Mineiro que estende `Agent`. Gerencia as variáveis de minério na mochila, estoque, desgaste da picareta e pedido do Ferreiro, iniciando diretamente no estado `Sleeping`.
4. **Ferreiro.java:** Classe do agente Ferreiro que estende `Agent`. Controla as variáveis de minério, progresso da espada, espadas prontas e ouro, mantendo a referência ao agente Mineiro e iniciando no estado `Sleeping`.
5. **Sleeping.java:** Estado compartilhado por qualquer agente quando está descansando.
6. **Mining.java:** Estado do Mineiro responsável por aumentar o minério na mochila e o desgaste da picareta, trocando para `Repairing` quando a mochila enche ou quando há um pedido do Ferreiro (Ou para `Sleeping`).
7. **Repairing.java:** Estado do Mineiro responsável por reduzir o desgaste da picareta e reiniciar o ciclo de mineração quando ela está nova.
8. **Forging.java:** Estado do Ferreiro que gasta minério e avança o progresso da espada a cada turno, fazendo as checagens para mudar para `Selling` ou `WaitingOre` (Ou para `Sleeping`).
9. **WaitingOre.java:** Estado de espera do Ferreiro que envia o pedido de minério ao Mineiro ao entrar e aguarda a confirmação de que há estoque disponível (Ou para `Sleeping`).
10. **Selling.java:** Estado do Ferreiro em que ele vende as espadas prontas, uma por hora, até acabarem, para então voltar a forjar ou esperar minério (Ou para `Sleeping`).

- 11. AgentManager.java:** Gerenciador da simulação que guarda a lista de agentes e executa o método update() de cada um em loop.
- 12. Main.java:** Ponto de entrada que cria os agentes, adiciona no AgentManager e inicia a execução do código.

7. RESULTADOS ESPERADOS

----- HORA 00:00 -----

===== FERREIRO =====

Dormindo...

Minério disponível para forjar: 0

Progresso da espada: 0

Espadas prontas: 0

Ouro: 0

===== MINEIRO =====

Dormindo...

Minério na mochila: 0

Minério guardado no estoque: 0

Desgaste da picareta: 0

Pedido do ferreiro: false

----- HORA 20:00 -----

===== FERREIRO =====

Vendendo...

Minério disponível para forjar: 8

Progresso da espada: 0

Espadas prontas: 0

Ouro: 20

Forja acesa!

O minério foi guardado no estoque para o ferreiro!

MINEIRO foi dormir...

----- HORA 05:00 -----

===== FERREIRO =====

Dormindo...

Minério disponível para forjar: 0

Progresso da espada: 15

Espadas prontas: 1

Ouro: 20

MINEIRO acordou!

Vamos minerar!